

Saveetha Institue Of Medical And Technical Sciences

CSA3712-Software Testing

CO2_AT3 Error Analysis Task

Written by:

G Bhumika

192312237

Electronics &Communication Engineering

QUESTIONS

Question 1. A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.

Question 2. A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.

Question 3. A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.

Question 4. A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.

Question 5. You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.

Question 6. A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.

Question 7. An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.

Question 8. A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.

Question 9. A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.

Question 10. You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.

ANSWERS

Answer 1: ATM Withdrawal Module (Black Box Reverse Engineering)

1.1 Understanding the system and the observed behaviour

The ATM withdrawal module accepts a withdrawal amount from a customer and decides whether to dispense cash or show an error message. Because only inputs and outputs are visible, the internal rules have to be guessed from what the module does. The following input–output behaviour is assumed to have been observed (amounts in ₹):

Amount	Balance / Today's total	Observed output
500	Balance 10,000	Success, cash dispensed
10,000	Balance 50,000	Success, cash dispensed
10,100	Balance 50,000	"Exceeds per-transaction limit"
0 or -100	Balance 10,000	"Invalid amount"
250	Balance 10,000	"Enter amount in multiples of 100"
5,000	Balance 3,000	"Insufficient balance"
5,000	Balance 5,000	Success, balance becomes 0
2,000	Balance 20,000, already withdrawn today 24,000	"Daily limit exceeded"

1.2 Rules inferred from the behaviour

- Amount must be a positive number and a multiple of ₹100, with a minimum of ₹100.
- Maximum amount for one transaction is ₹10,000.
- Account balance must be greater than or equal to the amount (an equal balance is allowed).
- Total withdrawn in one day, including the current request, must not exceed ₹25,000.
- Every violated rule produces its own error message, so each rule is checked separately.

1.3 Decomposition into functional components

- **Amount input handler:** accepts the entered value and checks that it is numeric and positive.
- **Denomination validator:** checks that the amount is a multiple of 100 and not below the minimum.
- **Limit checker:** applies the per-transaction limit (₹10,000) and the daily limit (₹25,000).
- **Balance validator:** compares the amount with the available balance.

- **Cash availability check:** confirms that the ATM itself has enough notes to pay the amount.
- **Transaction processor:** debits the account, operates the dispenser and prints the receipt.
- **Message handler:** chooses the correct error or success message for the user.

1.4 Equivalence classes and boundary conditions

Input	Valid classes	Invalid classes	Boundary values
Amount	100 ≤ amount ≤ 10,000 and a multiple of 100	Amount ≤ 0; not a multiple of 100; amount > 10,000; non-numeric or blank; decimal	0, 99, 100, 200, 9,900, 10,000, 10,100
Balance	Balance ≥ amount	Balance < amount	Balance = amount, balance = amount − 1
Daily total	Earlier total + amount ≤ 25,000	Earlier total + amount > 25,000	Total = 25,000 and total = 25,100
ATM cash	Cash in machine ≥ amount	Cash in machine < amount	Cash = amount, cash = amount − 100

1.5 Black box techniques inferred

From the observations, the earlier testers most likely used **Equivalence Class Partitioning**, because there is one representative test for each error message (invalid amount, wrong denomination, insufficient balance, daily limit). They also used **Boundary Value Analysis**, but only partly: the values 10,000 and 10,100 and the "balance equals amount" case show boundary thinking, but similar boundaries for the minimum amount and the daily limit are missing. The negative amount and zero amount tests indicate **Error Guessing**. There is no sign of decision table testing or state-based testing, so combinations of rules and multi-step behaviour were probably not tested.

1.6 Evaluation of adequacy

The existing test set is adequate for single-rule checks but weak for everything else. The main gaps are:

- Boundaries are checked for only two of the four rules (per-transaction limit and balance). The minimum amount, the daily limit and the ATM cash level are not tested at their edges.
- No combination tests exist, for example an amount that breaks both the per-transaction limit and the balance rule. The order in which messages are shown is therefore unknown.
- Invalid data types such as letters, decimals and a blank entry are not covered.
- Hardware and failure scenarios are missing: ATM out of cash, dispenser failure after the account is debited, and network failure.

- Multi-transaction behaviour is untested: several withdrawals in a day until the limit is reached.

1.7 Reconstructed (improved) set of test cases

ID	Scenario	Input	Expected result	Technique
TC01	Minimum valid amount	Amt 100, bal 5,000	Success; balance 4,900	BVA
TC02	Just below minimum	Amt 99	"Multiples of 100" message	BVA
TC03	Zero amount	Amt 0	"Invalid amount"	BVA
TC04	Negative amount	Amt -100	"Invalid amount"	ECP
TC05	Typical valid amount	Amt 2,500, bal 20,000	Success; balance 17,500	ECP
TC06	Not a multiple of 100	Amt 250	"Multiples of 100" message	ECP
TC07	Per-transaction maximum	Amt 10,000, bal 50,000	Success	BVA
TC08	Just above maximum	Amt 10,100, bal 50,000	"Exceeds per-transaction limit"	BVA
TC09	Just below maximum	Amt 9,900, bal 50,000	Success	BVA
TC10	Non-numeric entry	Amt "abc"	"Invalid input"; no debit	ECP / error guessing
TC11	Blank entry	Amt blank, press Enter	"Invalid input"; no debit	Error guessing
TC12	Decimal amount	Amt 500.50	"Invalid amount"	ECP
TC13	Balance equals amount	Amt 5,000, bal 5,000	Success; balance 0	BVA
TC14	Balance one less than amount	Amt 5,000, bal 4,999	"Insufficient balance"	BVA
TC15	Zero balance	Amt 100, bal 0	"Insufficient balance"	ECP
TC16	Daily total reaches exactly the limit	Amt 1,000, earlier 24,000	Success	BVA
TC17	Daily total exceeds limit by 100	Amt 1,100, earlier 24,000	"Daily limit exceeded"	BVA
TC18	Per-transaction OK, daily limit broken	Amt 10,000, earlier 20,000	"Daily limit exceeded"	Combination
TC19	Two rules broken together	Amt 10,100, bal 5,000	One message in the specified order; no debit	Combination
TC20	ATM has less cash than amount	Amt 5,000, ATM cash 3,000	"Cannot dispense"; account not debited	ECP
TC21	ATM cash equals amount	Amt 5,000, ATM cash 5,000	Success	BVA
TC22	Dispenser fails after debit	Amt 2,000, simulated hardware fault	Amount re-credited automatically	Failure testing

ID	Scenario	Input	Expected result	Technique
TC23	Repeated withdrawals in one day	Ten withdrawals of 2,500	Tenth is rejected at the daily limit	Sequence test

1.8 Conclusion

By studying the visible input–output behaviour, the hidden rules of the ATM (limits, denominations, balance and daily total) were reconstructed and split into seven functional components. The original testing relied mostly on equivalence classes with a few boundary checks. The improved set of 23 test cases adds complete boundary coverage, combination tests and failure scenarios, so all critical withdrawal scenarios are now covered.

Answer 2: White Box Reverse Engineering using Control Flow Graph

2.1 Assumed code and execution traces

The module processes a list of n transaction amounts, ignores invalid (zero or negative) amounts, adds up the valid ones and reports whether the total crosses a limit. The line numbers below are used as node numbers in the control flow graph.

```
1   total = 0; i = 0; invalid = 0;
2   while (i < n) {
3       if (amt[i] <= 0)
4           invalid++;
        else
5           total = total + amt[i];
6       i++;
    }
7   if (total > limit)
8       status = "EXCEEDED";
    else
9       status = "WITHIN LIMIT";
10  return status;
```

Only two execution traces are available and no test documentation:

Trace	Input	Nodes executed	Result
T1	n = 3, amt = [100, 200, 300], limit = 1000	1, 2, (3, 5, 6) three times, 2, 7, 9, 10	WITHIN LIMIT
T2	n = 2, amt = [600, 700], limit = 1000	1, 2, (3, 5, 6) twice, 2, 7, 8, 10	EXCEEDED

2.2 Control flow graph

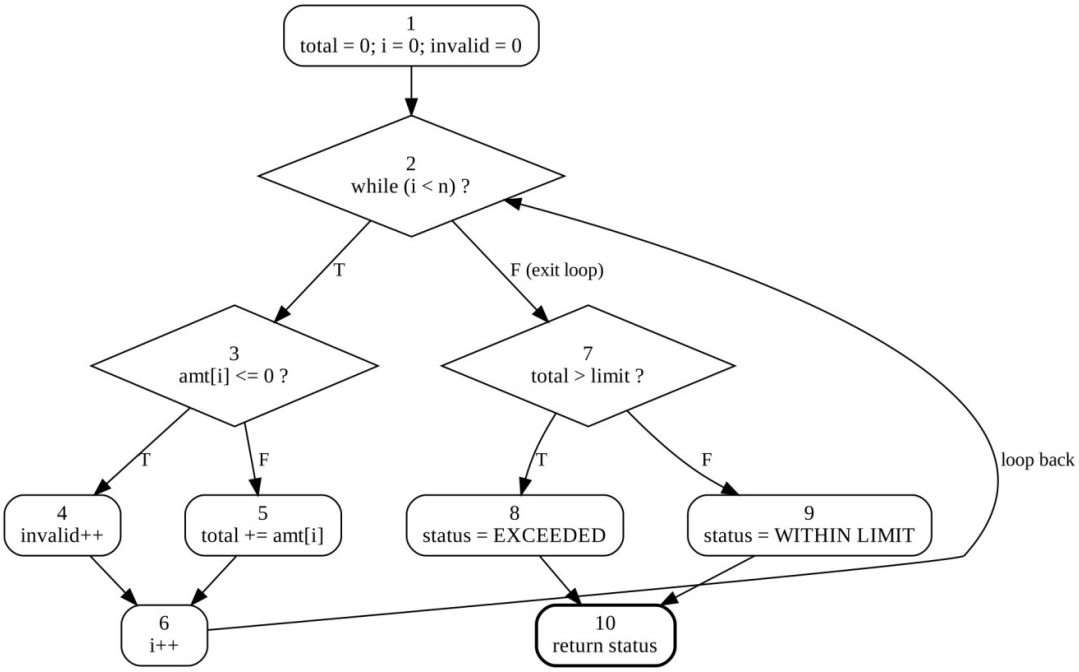


Figure 1: Control flow graph of the transaction module

2.3 Decision points and loops

- **Decision points:** node 2 (loop condition i < n), node 3 (amount ≤ 0) and node 7 (total > limit). So P = 3.
- **Loop:** one simple, non-nested while loop. Its body is nodes 3 to 6, and the back edge is 6 → 2.
- **Branch outcomes:** each decision has a true and a false outcome, giving 6 branch outcomes in total.

2.4 Cyclomatic complexity and independent paths

The number of independent paths is given by the cyclomatic complexity. The graph has N = 10 nodes and E = 12 edges.

- V(G) = E − N + 2 = 12 − 10 + 2 = **4**
- V(G) = P + 1 = 3 + 1 = **4**
- V(G) = number of enclosed regions + 1 = 3 + 1 = **4**

All three methods give 4, so there are four independent paths:

Path	Route (nodes)	Condition	Test data	Expected
P1	1-2-7-9-10	Loop skipped (n = 0)	n = 0, limit = 1000	WITHIN LIMIT
P2	1-2-3-4-6-2-7-9-10	One invalid amount	n = 1, amt = [-50]	WITHIN LIMIT; invalid = 1
P3	1-2-3-5-6-2-7-9-10	Valid amount, below limit	n = 1, amt = [400]	WITHIN LIMIT

Path	Route (nodes)	Condition	Test data	Expected
P4	1-2-3-5-6-2-7-8-10	Valid amount, above limit	n = 1, amt = [1200]	EXCEEDED

2.5 Adequacy of the existing tests

Coverage is calculated as (items executed ÷ total items) × 100. Traces T1 and T2 together execute every node except node 4 (invalid++), because both traces contain only valid positive amounts.

Coverage criterion	Covered	Total	Percentage
Statement (node) coverage	9	10	90%
Branch (decision) coverage	5	6	83.3%
Edge coverage	10	12	83.3%
Basis path coverage	2 (P3, P4)	4	50%
Loop coverage (zero, one, many iterations)	1 (many)	3	33.3%

The tests are therefore only partly adequate. The invalid-amount branch (3 → 4) is never executed, the loop is never skipped or run exactly once, and the boundary case where the total is exactly equal to the limit is never tried. A defect in the invalid-amount handling would go completely unnoticed.

2.6 Reconstructed path testing strategy

The strategy has four parts: (1) basis path testing for the four independent paths, (2) loop testing with zero, one, two, typical and maximum iterations, (3) boundary testing of the decision at node 7, and (4) a mixed test with both valid and invalid amounts.

TC	Input	Path / purpose	Expected result
TC1	n = 0, limit = 1000	P1, loop executed zero times	WITHIN LIMIT
TC2	n = 1, amt = [-50]	P2, invalid branch	WITHIN LIMIT; invalid = 1
TC3	n = 1, amt = [400], limit = 1000	P3, loop executed once	WITHIN LIMIT
TC4	n = 1, amt = [1200], limit = 1000	P4, limit exceeded	EXCEEDED
TC5	n = 2, amt = [100, -20], limit = 1000	Loop twice, both branches of node 3	WITHIN LIMIT; invalid = 1
TC6	n = 3, amt = [300, 300, 400], limit = 1000	Boundary: total = limit	WITHIN LIMIT
TC7	n = 3, amt = [300, 300, 401], limit = 1000	Boundary: total = limit + 1	EXCEEDED

TC	Input	Path / purpose	Expected result
TC8	n = 1, amt = [0], limit = 1000	Boundary of amt ≤ 0	WITHIN LIMIT; invalid = 1
TC9	n = 100, each amt = 10, limit = 1000	Loop at maximum size, total = limit	WITHIN LIMIT

After adding these tests the coverage becomes: statement 10/10 (100%), branch 6/6 (100%), edge 12/12 (100%), basis paths 4/4 (100%) and all loop cases (zero, one, many) covered.

2.7 Conclusion

The control flow graph shows three decision points, one loop and four independent paths. The existing traces reach only 90% statement and 83% branch coverage, and never test an empty loop or an invalid amount. The reconstructed nine-test strategy covers every statement, branch, basis path and loop condition, so the module is fully path-tested.

Answer 3: Web Application Login and Transaction System

3.1 Assumed defect log

Only defect logs and test results exist, so the testing technique has to be guessed from the type of defects that were found. The following defect log is assumed:

ID	Module	Defect	Revealed by
D1	Login	A 7-character password is accepted although the minimum is 8	Test at length 7
D2	Login	Account is locked after the 6th wrong attempt instead of the 5th	Test at attempts 5 and 6
D3	Login	A username containing a single quote shows a server error page	Special-character input
D4	Login	Empty password shows “Invalid username” instead of “Password required”	Blank field test
D5	Session	Transaction page opens after logout when the Back button is pressed	Navigation flow test
D6	Transaction	Transfer of exactly ₹1,00,000 (the limit) is rejected	Test at the limit
D7	Transaction	Transfer ₹1 above the balance is accepted	Test at balance + 1
D8	Transaction	Double-clicking “Pay” creates two debits	Repeated action
D9	Transaction	After a wrong OTP, the retry succeeds without a fresh OTP	Failure-then-retry flow

3.2 Defect pattern analysis and likely techniques

Defect pattern	Defects	Technique likely used
Defects exactly at a limit (length 7 vs 8, attempt 5 vs 6, amount = limit, balance + 1)	D1, D2, D6, D7	**Black box:** Boundary Value Analysis
Wrong handling of invalid or special data (quote character, empty field)	D3, D4	**Black box:** Equivalence Partitioning (invalid classes) and Error Guessing
Wrong behaviour in particular flows (logout then Back, wrong OTP then retry)	D5, D9	**White box:** branch / decision coverage of the session and OTP logic
Duplicate action on the same request	D8	Exploratory testing; no systematic technique

So the original team most likely used equivalence partitioning, boundary value analysis and error guessing on the black box side, and statement/branch coverage on the white box side. The white box testing looks shallow because the flow defects (D5, D9) were found late and only in two places.

3.3 Decomposition into testable components

- **Login / authentication:** username and password validation, attempt counter, account locking.
- **Session management:** session creation, timeout, logout, Back-button behaviour.
- **Transaction input validation:** amount format, range and recipient details.
- **Business rules:** balance check, transaction limit, OTP verification.
- **Transaction processing:** debit and credit, database update, duplicate-request protection.
- **Confirmation and audit log:** receipt, message and log entry for every transaction.

3.4 Gaps in the testing coverage

- Boundary values are checked on one side only (7 and 6) but not on the matching valid side or the maximum password length.
- The OTP (wrong, expired, blank) and the session timeout have no equivalence classes at all.
- Path testing was not planned: the logout and OTP-retry defects appeared only by chance.
- No combination tests exist, for example a locked account trying to start a transaction.
- Duplicate submission, simultaneous sessions and slow-network behaviour are untested.

3.5 Reconstructed testing strategy

Step 1: Equivalence Partitioning. Each input is divided into valid and invalid classes, and one test is taken from every class.

Input	Valid classes	Invalid classes
Username	6 to 20 alphanumeric characters	Empty; fewer than 6; more than 20; special or SQL characters
Password	8 to 16 characters with required mix	Empty; fewer than 8; more than 16; wrong password
OTP	Correct 6-digit OTP within validity time	Wrong OTP; expired OTP; blank; fewer or more than 6 digits
Amount	1 to 1,00,000 and not more than balance	Zero or negative; above 1,00,000; above balance; non-numeric

Step 2: Boundary Value Analysis. Values just below, on and just above each boundary are tested.

Field	Boundary values to test
Username length	5, 6, 7, 19, 20, 21
Password length	7, 8, 9, 15, 16, 17
Failed login attempts	4, 5, 6 (lock must happen on the 5th)
Amount	0, 1, 2, 99,999, 1,00,000, 1,00,001
Amount against balance	Balance − 1, balance, balance + 1

Step 3: Path testing. The login logic is written as steps so that the paths can be counted.

1. Read username and password.
2. If either is empty, show “required” and stop.
3. If the username is not found, show “invalid credentials” and stop.
4. If the account is locked, show “account locked” and stop.
5. If the password is wrong, increase the attempt counter; if the counter reaches 5, lock the account; show the error and stop.
6. Otherwise reset the counter, create a session and open the dashboard.

There are five decisions (empty fields, user found, locked, password correct, counter ≥ 5), so V(G) = 5 + 1 = 6 independent paths:

Path	Route	Test data	Expected
P1	Empty field	Username given, password blank	“Password required”
P2	Unknown user	Username not registered	“Invalid credentials”
P3	Locked account	Valid user already locked	“Account locked”
P4	Wrong password, counter below 5	Valid user, wrong password once	Error; counter = 1
P5	Wrong password, 5th attempt	Four earlier failures, fifth wrong	Account locked

Path	Route	Test data	Expected
P6	Correct login	Valid username and password	Dashboard opens; counter reset

The transaction flow has four decisions (amount valid, within limit, balance enough, OTP correct), so five basis paths are tested: invalid amount, above limit, insufficient balance, wrong OTP, and a fully successful transaction. In addition, two flow tests target the defects found earlier: pressing Back after logout (D5) and double-clicking "Pay" (D8).

3.6 Justification

The reconstructed strategy matches each defect category with the technique that is able to find it. Boundary defects (four out of nine) need BVA, invalid-data defects need equivalence classes, and flow defects need path testing. Using the three techniques together removes the weaknesses of each one: equivalence partitioning keeps the number of tests small, BVA catches the errors that cluster at the edges, and path testing makes sure every branch of the login and OTP logic is executed. Adding flow tests for duplicate submission and session handling closes the gaps that the earlier team found only by accident.

Answer 4: Legacy Inventory Management System

4.1 Assumed transaction logs

```
[OK]   UPD item=A101 stock=50  change=-10  new=40
[OK]   UPD item=A101 stock=40  change=-30  new=10   ALERT=LOW_STOCK
[OK]   UPD item=B202 stock=25  change=-14  new=11   (no alert)
[OK]   UPD item=B202 stock=11  change=-1   new=10   ALERT=LOW_STOCK
[OK]   UPD item=C303 stock=100 change=+400 new=500
[FAIL] UPD item=A101 stock=10  change=-11  reason=INSUFFICIENT_STOCK
[FAIL] UPD item=D404 stock=0   change=-1   reason=INSUFFICIENT_STOCK
[FAIL] UPD item=E505 stock=20  change=0    reason=INVALID_QTY
[FAIL] UPD item=F606 stock=900 change=-501 reason=INVALID_QTY
[FAIL] UPD item=G707 stock=1000 change=+1  reason=EXCEEDS_CAPACITY
[FAIL] UPD item=Z999 reason=ITEM_NOT_FOUND
```

4.2 Business logic inferred from the logs

- New stock = old stock + change. A positive change is a receipt and a negative change is an issue.
- A change of 0 is rejected as INVALID_QTY.
- The size of one change cannot exceed 500 (change −501 failed even though the stock was 900).
- The new stock cannot be below 0 (stock 10 with change −11 failed).
- The new stock cannot be above 1000, which is the item capacity (stock 1000 with +1 failed).

- A low-stock alert is raised when the new stock is 10 or less. The pair of logs “new = 11, no alert” and “new = 10, alert” shows that the threshold is exactly 10.
- An unknown item id is rejected with ITEM_NOT_FOUND.

4.3 Equivalence partitions and hidden boundary conditions

Parameter	Partitions	Boundary values
Change quantity	Valid issue (−1 to −500); valid receipt (+1 to +500); zero; more than 500 in size; non-numeric or decimal	−501, −500, −1, 0, +1, +500, +501
Resulting stock	Below 0 (fail); 0 to 1000 (pass); above 1000 (fail)	−1, 0, 1000, 1001
Low-stock alert	New stock 0 to 10 (alert); new stock 11 or more (no alert)	10, 11, and 0
Item id	Existing item; non-existing item; blank	Not applicable (valid / invalid only)

4.4 Decomposition of the update process into functional units

1. **Request validation:** checks item id and the change quantity (non-zero, numeric, size ≤ 500).
2. **Stock calculation:** computes the new stock from the old stock and the change.
3. **Rule check:** confirms that the new stock is between 0 and 1000.
4. **Persistence:** writes the new stock to the database.
5. **Alert generation:** raises LOW_STOCK when the new stock is 10 or less.
6. **Logging:** writes the success or failure record.

4.5 Gaps found in the logs

- A successful update that brings the stock exactly to 0 never appears; only failures at 0 are logged.
- The capacity boundary of exactly 1000 and the receipt size of exactly +500 and +501 are never seen.
- The alert is shown only for decreasing updates. It is not known what happens when stock is already low and a small receipt keeps it low, or when a receipt lifts it above 10.
- The order of checks is unknown when two rules are broken together.
- Decimal and non-numeric quantities, and two users updating the same item at the same time, are never recorded.

4.6 Reconstructed black box test design

TC	Stock, change	Expected	Justification
TC1	50, −10	Success, new 40, no alert	Normal valid partition
TC2	20, −10	Success, new 10, alert	Alert boundary (exactly 10)

TC	Stock, change	Expected	Justification
TC3	20, −9	Success, new 11, no alert	Alert boundary + 1
TC4	20, −11	Success, new 9, alert	Just below the alert threshold
TC5	10, −10	Success, new 0, alert	Zero-stock boundary (missing in logs)
TC6	10, −11	INSUFFICIENT_STOCK	New stock = −1
TC7	0, −1	INSUFFICIENT_STOCK	Issue from empty stock
TC8	5, +10	Success, new 15, alert cleared	Receipt that moves stock out of the alert range
TC9	900, +100	Success, new 1000	Capacity boundary (missing in logs)
TC10	1000, +1	EXCEEDS_CAPACITY	Capacity boundary + 1
TC11	600, +500	EXCEEDS_CAPACITY	Valid size but capacity broken (new 1100)
TC12	900, −500	Success, new 400	Maximum change size
TC13	900, −501	INVALID_QTY	Maximum change size + 1 (issue)
TC14	100, +501	INVALID_QTY	Maximum change size + 1 (receipt)
TC15	20, 0	INVALID_QTY	Zero change
TC16	Unknown item, −5	ITEM_NOT_FOUND	Invalid item partition
TC17	20, 2.5 and “abc”	INVALID_QTY	Invalid data type
TC18	10, −600	One error, in the defined order	Two rules broken; finds the check order
TC19	Two users each issue −6 from stock 10	One succeeds, one fails; stock never below 0	Concurrency (gap in logs)

4.7 Justification of the selected test cases

Each test case is chosen from the inferred logic. TC2 to TC5 test the alert threshold from both sides, because the logs show only the 10/11 pair for decreasing stock. TC5, TC9 and TC10 test the boundary values of the stock range (0 and 1000), which the logs never reached. TC12 to TC15 cover the size rule on both sides and for both receipts and issues. TC8 and TC17 to TC19 cover behaviours that the logs do not show at all: alert clearing, wrong data types, rule order and concurrent updates. Using equivalence partitions to choose the classes and boundary values to choose the exact numbers keeps the suite short (19 tests) while covering every rule that was inferred.

Answer 5: Library Management Module (.exe) with CPU/Memory Profile

5.1 Understanding the system and the profile

Here only a compiled program and a resource profile are available, so source-level analysis is not possible. The idea is to treat CPU and memory spikes as clues about loops and decisions inside the code, because a loop that runs more often costs more CPU, and a loop that keeps allocating memory makes memory grow. The following stress-test profile is assumed:

No.	Observation in the profile	Inferred construct in the code
1	CPU grows in a straight line as the catalogue grows from 1,000 to 10,000 records	A single loop that scans every catalogue record (Loop 1)
2	CPU grows much faster than the catalogue size (roughly four times when the catalogue is doubled) for books with many copies	A nested loop: for every matching record, another loop goes through all copies (Loop 2)
3	Short, equal-sized CPU bursts for every issue request	A decision cluster (member valid, limit reached, fine pending) evaluated once per request
4	CPU burst proportional to the number of overdue days for some members	A fine-calculation loop that runs once per overdue day (Loop 3)
5	Memory rises steadily during repeated searches and never comes down	Memory allocated inside a loop and not released, or an in-memory list that keeps growing
6	At peak load the response time rises and the program occasionally freezes	A heavy path that is reached by many users at once

5.2 White box testing requirements derived

- **Loop testing** for all three loops: zero, one, two, typical, maximum and maximum + 1 iterations.
- **Nested loop testing** for Loop 2, keeping the outer loop at its minimum while testing the inner loop and then the reverse.
- **Decision and condition coverage** for the decision cluster, so that every combination of “limit reached” and “fine pending” is exercised.
- **Path coverage** of the resource-intensive paths, together with measurement of CPU, memory and response time.

5.3 Conceptual control flow graph

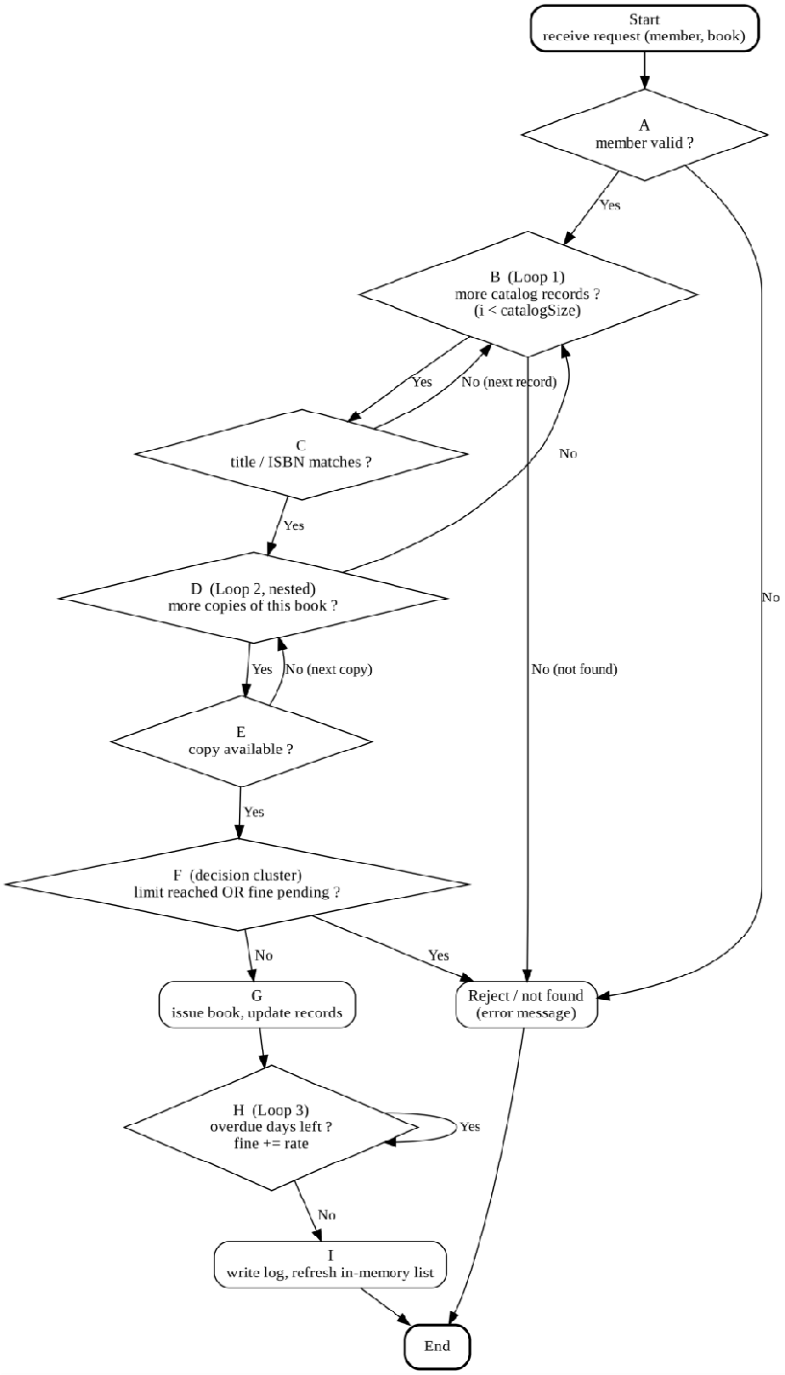


Figure 2: Conceptual CFG of the library issue-book module (reconstructed from the profile)

The graph has 12 nodes (Start, A to I, Reject and End) and 18 edges, so V(G) = E − N + 2 = 18 − 12 + 2 = 8. Seven decision nodes (A, B, C, D, E, F, H) give the same value, P + 1 = 8. If the two conditions inside F are counted separately, the value becomes 9. The loops are: B (scan of the catalogue), D nested inside B (scan of the copies), and H (day-by-day fine calculation).

5.4 Reconstructed path testing strategy for resource-intensive paths

ID	Path	Why it is heavy	Test	Measure
RP1	Start-A-B (all records)-Reject	Whole catalogue scanned, no match found	Title not present; catalogue of 10, 1,000, 10,000 and 1,00,000 records	CPU, time
RP2	A-B-C-D-E loops back to D and B	Nested loop: many copies, none available	Book with 1, 10 and 500 copies, all on loan	CPU growth shape
RP3	A-B-C-D-E-F-G-H-I	Fine loop runs for every overdue day	Overdue 0, 1, 30, 365 and 3,650 days	CPU, time
RP4	A-…-F-Reject	Decision cluster	Limit reached only; fine pending only; both; neither	Correct message
RP5	Full success path repeated	Possible memory leak	Issue and return the same book 10,000 times	Memory trend
RP6	RP1 and RP2 run together	Concurrent load	50 users searching at the same time	CPU, freeze

5.5 Impact on system stability

Targeting the heavy paths shows how the program behaves under load before real users are affected. The expected benefits are:

- The growth curve of RP1 and RP2 shows whether the cost is linear or quadratic. A quadratic curve points to the nested loop as the main bottleneck, which can be fixed by indexing or a hash lookup.
- RP5 shows whether memory returns to its starting level. A steadily rising line means a leak that would crash the program in a long-running session.
- RP3 checks that a very large overdue period cannot lock the CPU for one request.
- Acceptance limits can be defined, for example CPU below 80% for 95% of the time, response under 2 seconds, memory growth under 5% in an 8-hour run, and no freeze with 50 users.

One limitation should be mentioned. Because the source is not available, the mapping from spikes to loops is an inference. It should be confirmed with a disassembler or a profiler call graph, but even without that, the reconstructed paths give a clear, testable list of resource-intensive scenarios.

Answer 6: Mobile Payment Gateway State Machine

6.1 Assumed API responses

```
// Pending
{ "txn_id": "T1001", "status": "PENDING", "amount": 500.00, "settlement_id": null }
// Success
{ "txn_id": "T1001", "status": "SUCCESS", "amount": 500.00, "settlement_id": "S778", "code": "00" }
// Timeout
{ "txn_id": "T1002", "status": "TIMEOUT", "amount": 500.00, "settlement_id": null, "code": "408", "elapsed_ms": 30000 }
// Reversal
{ "txn_id": "R5001", "status": "REVERSAL", "original_txn_id": "T1001", "amount": 500.00, "reason": "CUSTOMER_REFUND" }
```

6.2 Inferred state machine

The responses show that the gateway behaves like a finite state machine. A transaction is always in exactly one state, and an event (a trigger) moves it to another state. The data constraints seen in the JSON also help to decide which transitions are allowed:

- **PENDING:** settlement_id must be null, because no money has been settled yet.
- **SUCCESS:** settlement_id must be present and the code is "00".
- **TIMEOUT:** settlement_id is null, the code is 408 and the elapsed time is at least the time-out limit T (30 seconds here).
- **REVERSAL:** has its own reversal id, refers to an original transaction through original_txn_id, and the amount must not be more than the original amount.

The legal transitions inferred are:

From	Trigger	To	Data constraint
Start	Initiate payment	PENDING	Amount > 0; settlement_id = null
PENDING	Bank confirms	SUCCESS	settlement_id present; code 00
PENDING	No response within T seconds	TIMEOUT	code 408; elapsed ≥ T
TIMEOUT	Status re-check / retry	PENDING	Same txn_id reused
TIMEOUT	Auto reversal (no confirmation)	REVERSAL	original_txn_id present; amount ≤ original
SUCCESS	Refund / reversal request	REVERSAL	original_txn_id present; amount ≤ original

6.3 State-transition diagram

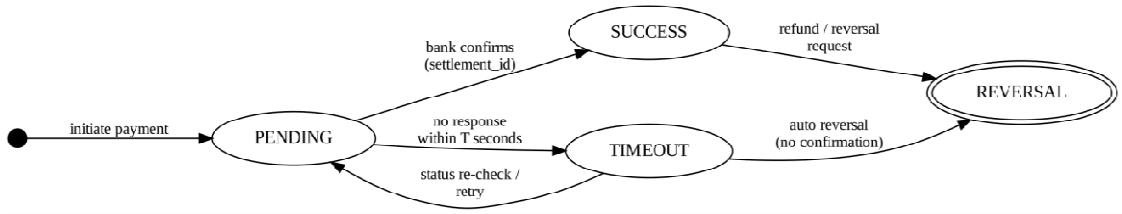


Figure 3: State-transition diagram of the payment gateway

In the diagram, the black dot is the start, ellipses are states, arrows are transitions labelled with the trigger, and the double-bordered state REVERSAL is the final state (no transition leaves it).

6.4 Implicit technique identified

The behaviour follows **State Transition Testing**, a black box technique. It models the system as states, events, transitions and actions. Coverage in this technique has levels: state coverage (every state visited), transition coverage or 0-switch coverage (every legal transition used at least once), and N-switch coverage (sequences of N + 1 transitions). It also includes negative testing, where an event that is not allowed in a state must be rejected. Since the JSON responses include fixed statuses and time-out behaviour, the gateway developers most likely designed the tests with this technique, but only with legal transitions.

6.5 State table (all legal and illegal transitions)

A state table lists every state against every event, which makes the illegal transitions visible. “Invalid” means the event is not allowed in that state.

State	Bank confirms	No response	Retry	Refund	Auto reversal
PENDING	SUCCESS	TIMEOUT	Invalid	Invalid	Invalid
SUCCESS	Invalid	Invalid	Invalid	REVERSAL	Invalid
TIMEOUT	Invalid	Invalid	PENDING	Invalid	REVERSAL
REVERSAL	Invalid	Invalid	Invalid	Invalid	Invalid

There are 4 states × 5 events = 20 cells. Five of them are legal (plus the start transition) and the other 15 are illegal.

6.6 Reconstructed state-based test suite

(a) Legal transitions (transition coverage)

ID	Start state	Event	Expected result
TL1	Start	Initiate payment	State = PENDING; settlement_id null
TL2	PENDING	Bank confirms	State = SUCCESS; settlement_id present
TL3	PENDING	No response for T seconds	State = TIMEOUT; code 408

ID	Start state	Event	Expected result
TL4	TIMEOUT	Status re-check / retry	State = PENDING
TL5	TIMEOUT	Auto reversal	State = REVERSAL; linked to original
TL6	SUCCESS	Refund request	State = REVERSAL; amount ≤ original

(b) Sequence tests (1-switch and longer paths)

- S1: Start → PENDING → SUCCESS → REVERSAL (complete life of a successful payment and refund).
- S2: Start → PENDING → TIMEOUT → PENDING → SUCCESS (recovery after a time-out).
- S3: Start → PENDING → TIMEOUT → REVERSAL (money returned after an unconfirmed payment).
- S4: Start → PENDING → TIMEOUT → PENDING → TIMEOUT (loop: time-out twice in a row).

(c) Illegal transitions (negative tests). In each case the gateway should reject the request with an error such as "409 Invalid state", leave the state unchanged and move no money.

ID	State	Event attempted	Expected result
TI1	PENDING	No response again / retry	Rejected; state stays PENDING
TI2	PENDING	Retry	Rejected
TI3	PENDING	Refund before success	Rejected; no reversal created
TI4	PENDING	Auto reversal	Rejected
TI5	SUCCESS	Bank confirms (duplicate)	Rejected; no second settlement
TI6	SUCCESS	No response (time-out)	Rejected; state stays SUCCESS
TI7	SUCCESS	Retry	Rejected
TI8	SUCCESS	Auto reversal	Rejected
TI9	TIMEOUT	Bank confirms directly	Rejected; must go through PENDING
TI10	TIMEOUT	No response again	Rejected
TI11	TIMEOUT	Customer refund	Rejected
TI12	REVERSAL	Bank confirms	Rejected; final state
TI13	REVERSAL	No response	Rejected
TI14	REVERSAL	Retry	Rejected
TI15	REVERSAL	Refund again (double reversal)	Rejected; no second refund

(d) Data-constraint tests. A reversal for more than the original amount must fail; a SUCCESS response without a settlement_id must be treated as an error; a REVERSAL that refers to a non-existing original_txn_id must be rejected.

The suite therefore covers all 4 states, all 6 legal transitions and all 15 illegal combinations.

Answer 7: Encrypted Communication Module

7.1 Fuzzing analysis of the input fields

Fuzzing means sending many unusual, extreme or random values to every input field and observing the output. The module is assumed to have three inputs: recipient_id (integer), message (text) and key_size (bits). The following results are assumed from the fuzzing run:

Fuzzed input	Observed response	Inference
message = empty	ERR_101 “Message required”	Minimum length is 1
message = 1 character, then 1024 bytes	Encrypted output returned	Both lengths accepted
message = 1025 bytes	ERR_102 “Message too long”	Maximum length is 1024 bytes
message = 65,536 bytes	ERR_102, no crash	Length is checked before processing
message = 2,000,000 bytes	Rejected after about 8 seconds	Weakness: size is checked late
message contains a null byte	Output shorter than expected	Possible truncation at the null byte
message = SQL-like text	Accepted and encrypted	Message is treated as plain data
recipient_id = 0, -1, 100000	ERR_103 “Out of range”	Valid range is 1 to 99999
recipient_id = “abc” or 1.5	ERR_104 “Wrong type”	Integer type is enforced
key_size = 100 or 512	ERR_105 “Unsupported key size”	Only 128, 192, 256 are allowed

7.2 Validation logic reconstructed

- recipient_id must be an integer between 1 and 99999.
- message must have between 1 and 1024 bytes.
- key_size must be one of 128, 192 or 256.
- The error code depends on the type of fault (101 required, 102 length, 103 range, 104 type, 105 key size), so the checks are done one by one in this order.
- Two fuzzing results point to possible weaknesses: the late rejection of a very large message and the odd output for a null byte.

7.3 Decomposition of the module

Data validation component. It receives the raw input and performs the type check, the range and length check, the character and encoding check, and the key-size whitelist check. Its output is either a clean, accepted set of values or an error code.

Encryption component. It takes only the validated values. It prepares the key, generates a fresh random initialisation vector (IV), pads the message to the cipher block size, encrypts it and encodes the output (for example as hexadecimal). Because the validation component stands in front of the encryption component, any weakness in validation can pass bad data to the encryption code.

7.4 Improved test cases using Boundary Value Analysis

ID	Field	Test value	Expected result
B01	message length	0 bytes	ERR_101
B02	message length	1 byte	Encrypted output
B03	message length	2 bytes	Encrypted output
B04	message length	512 bytes (nominal)	Encrypted output
B05	message length	1023 bytes	Encrypted output
B06	message length	1024 bytes	Encrypted output
B07	message length	1025 bytes	ERR_102
B08	recipient_id	0	ERR_103
B09	recipient_id	1	Accepted
B10	recipient_id	2	Accepted
B11	recipient_id	99998	Accepted
B12	recipient_id	99999	Accepted
B13	recipient_id	100000	ERR_103
B14	key_size	127, 129 and 257	ERR_105 for each
B15	key_size	128, 192, 256	Accepted; ciphertext length matches the cipher
B16	Encryption output	Same message encrypted twice	Different ciphertext (random IV)
B17	Encryption output	Decrypt the ciphertext	Original message is recovered
B18	Encryption output	Message of 16, 17 bytes (block size edges)	Padding correct; round trip passes

7.5 Tests for buffer overflow and injection

ID	Input	Expected result
OV1	Message of 1025 bytes of “A”	Rejected with ERR_102; no crash
OV2	Message of 4096 bytes and of 1 MB	Rejected quickly; memory stays normal
OV3	recipient_id with 20 digits	ERR_103 or ERR_104; no integer overflow
OV4	Message with 600 two-byte characters (1200 bytes)	Rejected; length counted in bytes, not characters
IN1	Message: ' OR '1'='1	Encrypted as plain data; no database effect
IN2	Message: <script>alert(1)</script>	Encrypted as plain data; nothing is executed
IN3	Message: %s%s%n%x	No memory content is printed; treated as text
IN4	Message with a null byte in the middle	Whole message kept or rejected consistently, never silently cut
IN5	Message with line-break and control characters	Handled as data; no log or header injection
IN6	recipient_id: 5; DROP TABLE users	ERR_104; request stops at validation

7.6 How the design prevents buffer overflow and injection

Boundary value analysis is the main tool against overflow, because overflow bugs appear exactly at the limit of a buffer. The tests at 1023, 1024 and 1025 bytes prove that the 1024-byte limit is enforced, and the larger payloads (4096 bytes, 1 MB) show that the check happens before the data is copied. The multi-byte test (OV4) shows that the length is measured in bytes, which is where many overflows come from. For injection, the tests confirm the following design rules:

- Every field is validated against a strict whitelist (type, range, length, allowed key sizes) before anything else happens, and invalid input is rejected, not "cleaned".
- Message text is treated only as data. It is never joined into a database query or command, so SQL, script and format-string payloads have no effect.
- Bounded memory functions are used with a length that has already been verified.
- Error messages are fixed codes and do not reveal internal details.

Together, the fuzzing results, the BVA tests and the injection tests give evidence that malformed input cannot reach the encryption code or the memory buffer.

Answer 8: Insurance Premium Calculator

8.1 Assumed code and legacy results

```
1  premium = 1000;
2  if (age < 25 || claims > 2)
3      premium = premium * 1.5;
4  if (smoker && age > 40)
5      premium = premium + 800;
6  if (premium > 2000)
7      premium = 2000;
8  return premium;
```

The legacy results contain one test, T1: age = 50, smoker = true, claims = 3. The premium is 1000 × 1.5 = 1500, then 1500 + 800 = 2300, which is capped to 2000. The expected value 2000 is returned, so the test passes.

8.2 Original approach and why it is inadequate

T1 executes every statement (lines 1 to 8), so the result is 100% statement coverage. This shows that the original white box approach was **statement coverage only**, which is the weakest coverage criterion. It only proves that each line ran once. It does not check the false outcome of any if, and it does not check each condition inside || and &&.

For the three decisions, T1 makes every one of them true, so branch coverage is only 3 out of 6 (50%). Among the five individual conditions, each one is tested with a single value only. The following faults all give the correct answer 2000 for T1, so they are not detected:

No.	Faulty code	Result for T1 (50, true, 3)	Detected?

No.	Faulty code	Result for T1 (50, true, 3)	Detected?
D1	claims > 2 written as claims >= 2	3 satisfies both: 2000	No
D2	age < 25 written as age <= 25	50 is false in both: 2000	No
D3	age > 40 written as age >= 40	50 is true in both: 2000	No
D4	smoker && age > 40 written as smoker || age > 40	Both true in both: 2000	No

8.3 New control flow graph

To see the individual conditions, the compound conditions are split, so age < 25 || claims > 2 becomes two nodes (2 and 3) and smoker && age > 40 becomes two nodes (5 and 6).

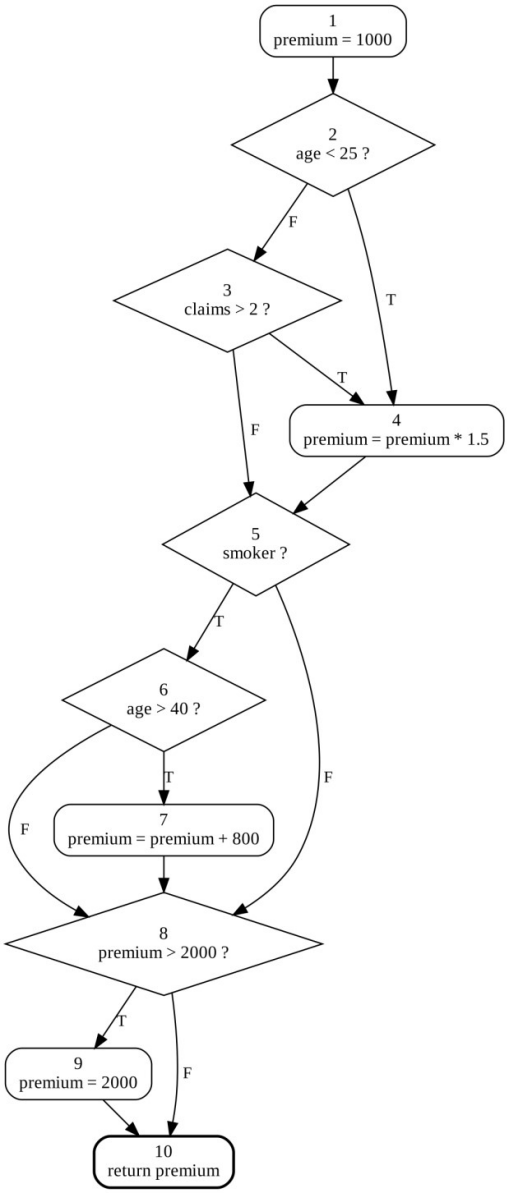


Figure 4: Condition-level CFG of the premium calculator

8.4 Cyclomatic complexity and independent paths

The graph has N = 10 nodes and E = 14 edges, so V(G) = E − N + 2 = 14 − 10 + 2 = 6. There are five predicate nodes (2, 3, 5, 6, 8), so V(G) = P + 1 = 6 as well. Therefore there are six independent (basis) paths:

Path	Route (nodes)	Condition
BP1	1-2-4-5-8-10	Age < 25; not a smoker; no cap
BP2	1-2-3-4-5-8-10	Age ≥ 25, claims > 2; not a smoker; no cap
BP3	1-2-3-5-8-10	Age ≥ 25, claims ≤ 2; not a smoker; no cap
BP4	1-2-3-5-6-8-10	No loading; smoker with age ≤ 40; no cap
BP5	1-2-3-5-6-7-8-10	No loading; smoker with age > 40; surcharge; no cap
BP6	1-2-3-4-5-6-7-8-9-10	Claims loading and surcharge; premium capped

8.5 Reconstructed path testing strategy

Looking at the full graph, the first decision has three routes (via age < 25, via claims > 2, or no loading), the second decision has three routes (non-smoker, smoker aged 40 or less, smoker over 40) and the cap has two routes, so there are 3 × 3 × 2 = 18 theoretical paths. Only **8 are feasible**. Two paths need age to be both below 25 and above 40, which is impossible, and the cap can only happen when the premium reaches 2300 (1500 + 800), so the remaining combinations of cap and no-cap are impossible. Tests TC1 to TC8 cover all eight feasible paths (and so all six basis paths), and TC9 to TC14 are boundary tests that expose the faults D1 to D4.

TC	Age	Smoker	Claims	Path	Expected	Purpose
TC1	20	No	0	BP1	1500	Age loading only
TC2	30	No	3	BP2	1500	Claims loading only
TC3	30	No	0	BP3	1000	No loading, no surcharge
TC4	30	Yes	0	BP4	1000	Smoker aged 40 or less
TC5	50	Yes	0	BP5	1800	Smoker surcharge only
TC6	50	Yes	3	BP6	2000	Loading, surcharge and cap
TC7	20	Yes	0	Extra path	1500	Age loading, smoker aged 40 or less
TC8	30	Yes	3	Extra path	1500	Claims loading, smoker aged 40 or less
TC9	24	No	0	BP1	1500	Boundary age 24
TC10	25	No	0	BP3	1000	Boundary age 25 (kills D2)
TC11	30	No	2	BP3	1000	Boundary claims 2 (kills D1)
TC12	40	Yes	0	BP4	1000	Boundary age 40 (kills D3)
TC13	41	Yes	0	BP5	1800	Boundary age 41
TC14	50	No	0	BP3	1000	Non-smoker aged over 40 (kills D4)

Coverage achieved: statement 8/8 (100%), branch 10/10 outcomes at condition level (100%), edge 14/14 (100%), basis paths 6/6 (100%) and feasible complete paths 8/8 (100%). A fifth fault, changing premium > 2000 to premium >= 2000, cannot change any result, since the premium before the cap is only ever 1000, 1500, 1800 or 2300. It is an equivalent change and not a real defect.

8.6 How this reduces the risk of calculation errors

- Every condition is tested with both outcomes and on both sides of its boundary, so small slips such as < instead of <= or || instead of && are caught.
- Every combination of loading, surcharge and cap is executed, so wrong interactions between rules (for example the cap being applied at the wrong step) are visible in the result.
- Each test has a calculated expected premium, so a wrong amount fails immediately instead of passing silently as in the legacy suite.
- Customers are not over- or under-charged, which lowers financial and regulatory risk for the insurance company.

Answer 9: Cloud-based File Synchronisation Tool

9.1 Assumed "lost file" defect reports

ID	Size	Type	Network	What happened
L1	2.3 GB	.mp4 video	Slow (0.5 Mbps)	File missing in the cloud after the sync
L2	4 GB	.zip backup	Average (8 Mbps)	Only part uploaded; tail missing
L3	0 KB	.txt	Fast	Empty file never synced
L4	Exactly 5 MB	.pdf	Fast	Last chunk missing; file corrupt
L5	120 MB	.docx	Dropped halfway	Upload not resumed; file lost
L6	15 KB	.tmp / lock files	Fast	Silently skipped
L7	800 MB	.psd	Slow (1 Mbps)	Session time-out before completion

9.2 Reverse engineering the synchronisation logic

The pattern in the reports says a lot about how the algorithm is built:

- **Size:** failures above 2 GB (L1, L2) suggest a 32-bit size field or a size limit, and the failure at exactly 5 MB (L4) suggests the file is uploaded in 5 MB chunks and the last chunk is dropped when the size is an exact multiple of the chunk size.
- **Size zero:** an empty file is skipped (L3), so the code probably uses "if size > 0" instead of "size ≥ 0".
- **Type:** temporary and lock files are filtered out by extension (L6), but without telling the user.
- **Network speed:** a fixed time-out that does not grow with file size means that a large file on a slow link cannot finish (L1, L7).

- **Interruption:** there is no reliable resume from the last good chunk (L5).

9.3 Equivalence classes likely used by the original developers

Factor	Classes the developers probably tested	Classes that were missed
File size	Small (below 1 MB) and medium (1 to 100 MB)	Zero byte, exact chunk size, large (100 MB to 2 GB), huge (above 2 GB)
File type	Documents (.docx, .pdf, .txt) and images	Large media, archives, system and temporary files
Network	Fast and normal speed	Slow, intermittent and offline-then-online

In other words, the developers tested only the comfortable middle of each factor. Every reported loss lies at an edge or in a class that was never tested.

9.4 Revised testing strategy: BVA integrated with stress testing

Boundary value analysis. Boundaries are tested for size (0 byte, 1 byte, chunk size − 1, chunk size, chunk size + 1, 100 MB ± 1 byte, 2 GB − 1, 2 GB, 2 GB + 1 and 4 GB), and for network speed (offline, 256 kbps, 1 Mbps, 10 Mbps) including the point at which the connection drops (at 1%, 50% and 99% of the upload).

Stress testing. The system is pushed beyond normal load:

- Sync 10,000 small files at once.
- 100 users syncing large files at the same time.
- Continuous sync for 24 hours with a throttled network.
- Disk almost full on the client or server.
- Repeated network drops and reconnections during one upload.

Verification of every test. After each sync, the checksum (for example SHA-256) of the source and the cloud copy must match, and the file count must be equal. Silent skipping is not allowed; a skipped file must be reported to the user.

ID	Size	Type	Network	Expected result
T1	0 byte	.txt	Fast	File synced (empty file exists in cloud)
T2	1 byte	.txt	Fast	Synced; checksum matches
T3	5 MB − 1 byte	.pdf	Fast	Synced; checksum matches
T4	Exactly 5 MB	.pdf	Fast	Synced; last chunk present
T5	5 MB + 1 byte	.pdf	Fast	Synced; two chunks, checksum matches
T6	2 GB − 1, 2 GB, 2 GB + 1 byte	.mp4	Average	All synced; no size error
T7	4 GB	.zip	Slow (1 Mbps)	Completes; time-out grows with size
T8	120 MB	.docx	Dropped at 50%	Resumes from the last good chunk

ID	Size	Type	Network	Expected result
T9	120 MB	.docx	Dropped at 99%	Resumes; no data lost
T10	15 KB	.tmp	Fast	Skipped with a visible notice to the user
T11	10,000 files of 10 KB	Mixed	Fast	All 10,000 present; no file missing
T12	100 users × 500 MB	Mixed	Average	No loss; no corruption under load
T13	800 MB	.psd	Slow (1 Mbps)	Completes without session time-out

9.5 How the approach prevents future data loss

- Boundary values (0 byte, chunk size, 2 GB) cover the exact places where the reported losses happened, so those defects cannot return unnoticed.
- Equivalence classes for all sizes, types and network speeds remove the blind areas of the earlier tests.
- Stress tests reveal losses that appear only under heavy load or long running time.
- Resume tests make sure that an interrupted upload continues from the last good part and does not restart or drop data.
- The checksum and file-count check turns every test into a clear pass or fail on the question that really matters: is the file in the cloud identical to the original?

Answer 10: E-commerce Search and Filter Module

10.1 Assumed SQL queries

```
Q1: SELECT * FROM products;
Q2: SELECT * FROM products WHERE in_stock = 1;
Q3: SELECT * FROM products WHERE category = 'Laptops';
Q4: SELECT * FROM products WHERE category = 'Laptops'
    AND price BETWEEN 30000 AND 60000;
Q5: SELECT * FROM products WHERE category = 'Laptops'
    AND brand IN ('Dell', 'HP');
Q6: SELECT * FROM products WHERE category = 'Laptops'
    AND brand IN ('Dell', 'HP') AND in_stock = 1;
Q7: SELECT * FROM products WHERE category = 'Laptops'
    AND NOT (brand = 'Acer') AND price BETWEEN 30000 AND 60000;
```

10.2 Search logic inferred

Inferred rule	Evidence
A filter that is not selected adds no condition at all	Q1 has no WHERE clause; Q2 contains only the in-stock condition
Different filters are joined with AND	Q4, Q6 and Q7 join category, price, brand and stock with AND

Inferred rule	Evidence
Several values of the same filter are joined with OR (implemented with IN)	Q5 and Q6: brand IN ('Dell', 'HP') means Dell OR HP
A price range is inclusive on both ends	Q4 and Q7 use BETWEEN
An excluded value is expressed with NOT	Q7: NOT (brand = 'Acer')
"In stock only" is a yes/no switch that adds in_stock = 1 only when ticked	Q2 and Q6 contain it; Q3 to Q5 do not

Thus the decision logic is: **(Category) AND (Price range) AND (Brand$_1$ OR Brand$_2$ …) AND (NOT excluded brand) AND (In stock)**, where each part is applied only when the user has selected it.

10.3 Decision table covering all filter combinations

Four filters can each be either "applied" (Y) or "not applied" (N): C1 = category, C2 = price range, C3 = brand, C4 = in-stock only. This gives $2^4 = 16$ rules, and every one of them is listed so that no combination is missed.

Rule	C1	C2	C3	C4	Expected query / result
R1	N	N	N	N	No WHERE clause (all products)
R2	N	N	N	Y	WHERE in_stock = 1
R3	N	N	Y	N	WHERE brand IN (?, ...)
R4	N	N	Y	Y	WHERE brand IN (?, ...) AND in_stock = 1
R5	N	Y	N	N	WHERE price BETWEEN ? AND ?
R6	N	Y	N	Y	WHERE price BETWEEN ? AND ? AND in_stock = 1
R7	N	Y	Y	N	WHERE price BETWEEN ? AND ? AND brand IN (?, ...)
R8	N	Y	Y	Y	WHERE price BETWEEN ? AND ? AND brand IN (?, ...) AND in_stock = 1
R9	Y	N	N	N	WHERE category = ?
R10	Y	N	N	Y	WHERE category = ? AND in_stock = 1
R11	Y	N	Y	N	WHERE category = ? AND brand IN (?, ...)
R12	Y	N	Y	Y	WHERE category = ? AND brand IN (?, ...) AND in_stock = 1
R13	Y	Y	N	N	WHERE category = ? AND price BETWEEN ? AND ?
R14	Y	Y	N	Y	WHERE category = ? AND price BETWEEN ? AND ? AND in_stock = 1
R15	Y	Y	Y	N	WHERE category = ? AND price BETWEEN ? AND ? AND brand IN (?, ...)
R16	Y	Y	Y	Y	WHERE category = ? AND price BETWEEN ? AND ? AND brand IN (?, ...) AND in_stock = 1

For each rule, the test checks two things: the generated SQL contains exactly the expected conditions, and every returned product satisfies all of them (and no matching product is left out).

10.4 Operator-level and special cases

The 16 rules show which filters are used, but not how the values inside them behave. These extra decision cases cover the AND/OR/NOT logic:

ID	Situation	Expected result
S1	Two brands selected (Dell, HP)	brand IN (Dell, HP): results are Dell or HP (OR inside one filter)
S2	Brand excluded (Acer)	NOT (brand = Acer): no Acer product appears
S3	Same brand both selected and excluded	Empty result or a clear “conflicting filters” message, not an SQL error
S4	Only minimum price given	price >= min
S5	Only maximum price given	price <= max
S6	Minimum price greater than maximum price	Validation message or empty result, not an SQL error
S7	All filters on, but no product matches	“No products found” message
S8	Filter value contains '; DROP TABLE products;--	Treated as text; zero results; no injection
S9	All filters cleared after use	Returns to rule R1: all products

10.5 Evaluation of the completeness of the model

- **Combination coverage:** all 16 of the 16 possible on/off combinations of the four filters are covered (100%).
- **Operator coverage:** AND between filters, OR inside one filter and NOT for exclusion are each tested, together with the boundary cases of the price range.
- **Traceability:** every inferred rule in section 10.2 is linked to at least one decision rule or special case.

The model has limits that should be stated honestly:

- It is built from only seven sample queries, so a filter that never appeared in the queries (for example rating or discount) is not in the model. Each extra filter doubles the table: six filters would need $2^6 = 64$ rules, and at that size pairwise (all-pairs) selection would be used to keep it manageable.
- The decision table checks which conditions are used, not every value. The values themselves need equivalence classes and boundary values (for example price 29,999 / 30,000 / 60,000 / 60,001).
- Sorting, pagination, keyword search and speed are outside this model.

- Because there is no source code, the inferred logic should be confirmed with the developers or with extra queries.

Within the observed filters, the reconstructed model is complete. Together with value-level testing and a check against the real product data, it gives a reliable test basis for the Search and Filter module.